

Chart of Accounts — System Blueprint

Audience: engineers working on anything that touches money. **Status:** current as at 2026-09-24, built from the live production schema and data. **Related:** [SYSTEM_CONTEXT.md](#) · [FINANCIAL_SYSTEM_ARCHITECTURE.md](#) · [RENT_COLLECTION_MONEY_MOVEMENT.md](#)

Contents

- [1. Read this first](#)
- [2. The posting model](#)
- [3. How to post](#)
- [4. Worked examples](#)
- [5. Account catalogue](#)
- [6. The resolution pipeline](#)
- [7. Controls](#)
- [8. Adding a new money flow](#)
- [9. Common mistakes](#)
- [10. Open defects](#)
- [11. Verification runbook](#)
- [12. Reference: category → account](#)
- [13. Glossary](#)

Terminology in user-facing copy. **Rent Plan** (never "loan"), **Supporter** / **Partner** (never "lender"), **Returns** (never "ROI" or "interest"). Internal category names still use the old words — do not propagate them into UI, SMS or email.

1. Read this first

Five things that surprise almost everyone:

- The ledger stores `direction`, never debit/credit.** The side is derived at read time from a mapping table. See [§2.2](#).
- `ledger_account_map` is not a lookup table — it is the accounting policy.** Changing one row re-states every historical leg that resolves through it.
- The balance sheet is not the sum of mapped legs.** A resolver applies shape-based overrides and injects synthetic legs that exist in no table. See [§6](#).
- Only `wallet`-scope legs move money.** `platform` and `bridge` legs are accounting-only. See [§2.4](#).
- Edge functions read `ledger_account_map` over PostgREST.** A database-only impact analysis will miss them and can block live money paths. See [§7.3](#).

2. The posting model

2.1 One table, three scopes

Everything posts to `general_ledger`. A row is a **leg**. Legs sharing a `transaction_group_id` form a **group**, and the group is the unit that must balance.

LEDGER_SCOPE	MEANING	MOVES WALLET BALANCES?
wallet	An effect on a named user's wallet	Yes
platform	The company-side counterpart	No
bridge	Recognition / restatement, no cash and no wallet effect	No

2.2 Direction, not debit/credit

```
side = (leg.direction = mapping.debit_when) ? DEBIT : CREDIT
```

debit_when is a property of the mapping row, not the leg. Two consequences:

- The same direction means different things for different categories.
- Flipping debit_when inverts every historical leg resolving through that row. Mapping edits are as consequential as data edits, and are not covered by the frontend ledger-write guard.

2.3 Wallet buckets

A wallet -scope leg carries a wallet_bucket :

BUCKET	MEANING	USER CAN WITHDRAW IT?
withdrawable	Spendable balance	Yes
float	Operating float held for trading	No — submit_withdrawal_request never reads float_balance
advance	Advance drawn against future earnings	No
locked	Restricted	No

2.4 The only path to a balance change

```
INSERT INTO general_ledger (ledger_scope = 'wallet', ...)  
|  
├─ trg general_ledger_route_buckets → tr_general_ledger_route_buckets()  
|   └─ returns immediately when ledger_scope <> 'wallet'  
|  
└─ apply_wallet_movement(user_id, category, amount, direction[, recipient_type])  
    |  
    ├─ wallets_physical (locked_balance)  
    └─ wallet_balances_projection (withdrawable / float / advance)  
        |  
        └─ wallets (VIEW over both)
```

ledger_account_map is not in this path. A mapping change can never move a balance — but it can block a posting, see §7.3.

3. How to post

3.1 The RPC

```
create_ledger_transaction(  
  entries          jsonb,          -- array of leg objects  
  idempotency_key  text  DEFAULT NULL,  
  skip_balance_check boolean DEFAULT false  
) RETURNS uuid          -- the transaction_group_id  
  
-- variant that posts into an existing group:  
create_ledger_transaction(  
  p_transaction_group_id uuid,  
  p_entries jsonb, p_idempotency_key text, p_skip_balance_check boolean  
) RETURNS uuid  
  
-- accrual-only (no wallet effect):  
create_ledger_transaction_accrual_only(entries jsonb) RETURNS jsonb
```

3.2 Leg shape

```
{  
  "ledger_scope": "wallet",          // wallet | platform | bridge  
  "direction": "cash_in",           // cash_in | cash_out  
  "amount": 20000,  
  "category": "agent_commission_earned",  
  "user_id": "...uuid...",          // REQUIRED on wallet scope  
  "wallet_bucket": "withdrawable",  // REQUIRED on wallet scope  
  "recipient_type": "user",          // user | operational_wallet  
  "source_table": "agent_collections", // provenance - always set it  
  "source_id": "...uuid...",  
  "reference_id": "TID...",          // external reference where one exists  
  "description": "Human-readable, states amounts and reason",  
  "currency": "UGX",  
  "transaction_date": "2026-09-24T09:00:00Z"  
}
```

Rules

- Always pass an `idempotency_key`. Retries are real; double-posting money is worse than failing.
- Always set `source_table`. Operator tools may legitimately leave `source_id` null (the description carries the narration), but a leg with neither is untraceable.
- `skip_balance_check` exists for legacy shapes. Do not reach for it.
- **Never write to `general_ledger` from the frontend.** Enforced at build time by `scripts/guard-frontend-ledger-writes.mjs`.
- **Never post to E4.** It is resolver-injected only.

3.3 Two gates every category must pass

1. `ledger_account_map` — an unmapped non-wallet leg resolves to **A9** and disappears from meaningful reporting.
2. `ledger_category_allowlist()` — `trg_validate_ledger_category` raises on any category outside it while `treasury_controls.strict_mode` is on (since 2026-04-09).

Mapped but not allowlisted → **fails at insert**. Allowlisted but not mapped → **posts and silently vanishes**.

4. Worked examples

Real groups from production. `raw` is what is written; `resolved` is what the balance sheet shows after §6.

4.1 Tenant collection — an agent collects rent with their float

Agent collects UGX 20,000 from a tenant; 10% commission.

Raw legs

SCOPE	BUCKET	CATEGORY	DIR	AMOUNT	RECIPIENT_TYPE
platform	—	tenant_repayment_collected	cash_in	20,000	—
wallet	float	agent_float_used_for_rent	cash_out	20,000	operational_wallet
platform	—	agent_commission_payable	cash_out	2,000	—
wallet	withdrawable	agent_commission_earned	cash_in	2,000	user

Resolved — the float leg triggers the `float_backed_collection_counterpart` synthetic and flips the repayment direction:

ACCOUNT	DR	CR	WHY
A5 Cash in Transit	20,000		<i>synthetic</i> — agent now holds the cash
L4 Landlord Rent Payable	20,000		<i>synthetic</i> — owed onward to the landlord
A3 Rent Access Receivables		20,000	tenant's Rent Plan balance reduces
A2 Float with Agents		20,000	agent's float was consumed
X3 Agent Commission Expense	2,000		commission earned
L1 Wallet Custody Payable		2,000	now owed to the agent
	42,000	42,000	✓

This is the most intricate journal in the system. The agent's own float funds the tenant's rent; the company recognises cash in custody and a landlord obligation.

4.2 Withdrawal — user takes money out

SCOPE	BUCKET	CATEGORY	DIR	AMOUNT	→
wallet	withdrawable	wallet_withdrawal	cash_out	200,000	DR L1
platform	—	wallet_withdrawal	cash_in	200,000	CR A1

Liability discharged, bank reduced. Balanced with no overrides.

4.3 Company float delivery — company sends float to an agent

SCOPE	BUCKET	CATEGORY	DIR	AMOUNT	→
wallet	float	agent_float_deposit	cash_in	3,000,000	DR A2
platform	—	agent_float_deposit	cash_out	3,000,000	CR A8

Float moves from the cycle-control account out to the agent.

4.4 Physical cash deposit → withdrawable (*post-fix shape*)

SCOPE	BUCKET	CATEGORY	DIR	→
platform	—	cash_receipt_in_transit	cash_in	DR A5
wallet	withdrawable	wallet_deposit	cash_in	CR L1

Cash received into custody, owed to the depositor. For a **float** deposit the shape differs — it adds `agent_float_cash_offset` (CR A2) and `cash_custody_payable` (CR L1) against a `agent_float_deposit` wallet leg (DR A2). Owned by `supabase/functions/approve-deposit/index.ts`; the `isFloatDeposit` / `needsPlatformOffset` flags decide which shape is written.

5. Account catalogue

Current assets

CODE	NAME	NOTES
A1	Cash and Bank Balances	Spendable now
A2	Cash at Hand — Float with Agents	⚠️ conflated — §10.1
A5	Cash in Transit — Received, Not Yet Banked	
A3	Rent Access Receivables (Tenants)	Rent Plan book
A4	Other Agent Receivables	⚠️ contaminated — §10.4
A6	Landlord Product Receivables	
A7	Partner Product Receivables	Promissory notes; ties exactly to subledger
A8	Agent and Merchant Float Cycle Control	Float clearing
A9	Suspense — Unresolved (debit)	Fallback for unmapped non-wallet legs
A10	Agent Advances Receivable	
A11	Agent Advance Access Fees Receivable	
A12	Merchandise and Smartphone Recovery Receivable	
A13	Bike Recovery Receivable	
A14	Credit Access Draws Receivable	
A15	Merchandise Credit Sales Receivable	Redundant with A12/A13; retirement pending
A16	Service Centre Advances Receivable	
A17	Service Centre Receivables	
A18	Tenant Service Charge Receivables	
A19	Business Advance Receivables	

Liabilities

CODE	NAME	NOTES
L1	Wallet Custody Payable (user balances)	Customer money; resolver-affected
L2	Partner Portfolios — Capital Held	Non-current
L3	Partner Returns / Rewards Payable	⚠️ never used — §10.5
L4	Landlord Rent Payable	Resolver-affected (synthetic debits)
L5	Agent Commission Payable	⚠️ nets to zero — §10.6
L6	Partner Top-Ups Awaiting Application	
L7	Platform Treasury Control — Landlord Flow	
L9	Suspense — Unresolved (credit)	No legs

Equity, revenue, expense

CODE	NAME	NOTES
E1	Shareholders' Capital Contributions	
E3	Legacy Opening Balance Adjustments	
E4	Legacy One-Sided Postings — Counterpart	Resolver-injected only — never post to it
R1	Platform Revenue	
X1	Operating Expenses	
X2	Partner Returns Expense	
X3	Agent Commission Expense	
X4	Credit Losses and Write-offs	
X5	Reimbursement of Agent-Fronted Payouts	
X6	Float Restatement Adjustments	

6. The resolution pipeline

`public.sofp_ledger_legs(as_at)` is canonical. `get_statement_of_financial_position()` is built on it.

1. FILTER `classification IN ('production','legacy_real')`
`OR (admin_correction AND source_table='merchant_float_reconciliations')`
2. BASE MAP `COALESCE(bucket-specific row, bucket-agnostic row, fallback)`
`fallback: wallet+float→A2 · wallet+advance→A4 · wallet→L1 · else→A9`
3. SHAPE `per-group counts: n_wallet, n_a2, n_l1, n_plat, n_repay, n_custody, ...`
4. OVERRIDES `account and direction rewritten by group shape`
5. SYNTHETICS `legs injected that exist in no table`
6. E4 `one-sided groups receive a balancing counterpart`

6.1 Which accounts base mapping can speak for

	ACCOUNTS
Safe — base mapping equals resolver	A5, A6, A7, A11, A16–A19, L2, L3, L5, L6, L7, L9
Unsafe — overrides/synthetics apply	A1, A2, A3, A4, A8, A10, A12–A15, L1, L4 , E3, X4, X6

Computing an unsafe account from `ledger_account_map` has produced wrong conclusions twice.

`get_treasury_cash_position()` is the one sanctioned exception: it computes A1 and A5 from base mapping deliberately and is authoritative for those two.

6.2 Account overrides

CONDITION	→
platform <code>system_balance_correction</code> , <code>n_a2>0</code> , src <code>merchant_float_reconciliations</code>	A8
platform <code>system_balance_correction</code> , <code>n_a2>0</code>	X6
platform <code>system_balance_correction</code>	E3
platform <code>agent_float_cash_offset</code> , src <code>agent_collections</code> , <code>cash_out</code>	A3
platform <code>rent_disbursement</code> , src <code>agent_advance_requests</code>	A10
<code>agent_repayment</code> / <code>advance_repayment</code> by source table	A10 · A12 · A13 · A14 · A15

CONDITION	→
platform <code>wallet_deduction%</code> , <code>n_a2>0</code> , <code>n_l1=0</code>	A1
platform <code>wallet_deduction%</code> , <code>n_l1>0</code>	E3
platform <code>wallet_withdrawal</code> , <code>n_wallet>0</code> , <code>n_l1=0</code>	L1
wallet leg at L1, not a bucket-move category, shape 2w / 1×A2 / 1×L1, no non-SBC platform legs	A8
platform <code>agent_float_cash_offset</code> , <code>n_custody>0</code> , <code>n_wallet>0</code> , <code>n_a2=0</code>	L1

Direction overrides mirror these. The most important flips `tenant_repayment*` / `rent_repayment` / `landlord_receivable_collected` when a float leg is present, so a collection **reduces** A3 instead of increasing it.

6.3 Synthetic legs

NAME	FIRES ON	EMITS
<code>float_backed_collection_counterpart</code>	platform <code>tenant_repayment*</code> / <code>rent_repayment</code> with a float leg	DR A5 + DR L4
<code>bucket_reclass_counterpart</code>	wallet L1 leg with <code>bucket_reclass_in/out</code> / <code>wallet_transfer</code> in a gated 2-leg shape	2 × X4 ⚠ <u>\$10.2</u>

6.4 E4 and its blind spot

Groups where `abs(raw_net) > 0.5` **and** DR ≠ CR receive a balancing **E4** leg.

Control gap. `raw_net` is raw `cash_in` - `cash_out`. A group with equal and opposite directions has `raw_net = 0` and is **exempt even when DR ≠ CR after mapping**. Every defect in §10 is of exactly this kind. **E4 being nil is not evidence that the books balance.**

7. Controls

7.1 Mapped-balance enforcement

`trg_enforce_ledger_group_mapped_balance` on `general_ledger`, **per ROW**.

- Judges only groups whose first leg is at/after `ledger_integrity_config.enforce_from` (**2026-07-23 19:30 UTC**)
- Computes DR/CR on **base mapping only** — no overrides, no synthetics
- `mapped_balance_mode = 'log'` → writes `ledger_mapped_balance_violations`
- Raises regardless of mode** if the group contains a treasury category: `treasury_fee_recognised`, `treasury_allocated`, `fee_receivable_created`, `partner_reward_accrued`, `agent_commission_accrued`, `treasury_net_revenue`

Because it ignores stages 4–6 it yields **false positives** against the published balance sheet. It also never clears a resolved violation — the table has no `resolved_at` column, unlike `critical_function_drift_alerts`.

7.2 Category allowlist

See §3.3.

7.3 `assertLedgerGroupBalanced` — the one that bites

`supabase/functions/_shared/balancedLedgerPost.ts` prices legs against `ledger_account_map` **over PostgREST** and refuses to write an unbalanced group ("assert-then-post: nothing is written when the assertion fails").

Callers — all cross-bucket money paths:

FUNCTION	TRAFFIC
<code>agent-convert-withdrawable-to-float</code>	~165 legs/week
<code>finops-wallet-move</code>	high
<code>admin-withdrawable-to-float</code>	low
<code>admin-float-to-withdrawable</code>	low

A `pg_proc` scan cannot see these. On 2026-09-24 a mapping change passed every database-side check and would have rejected all four paths; it was caught and rolled back 30 minutes later. Always `grep -rn "<category>" supabase/functions/`.

7.4 Drift and invariants

CONTROL	PURPOSE
<code>assert_money_path_intact()</code>	17 invariants over the collection/allocation path
<code>critical_function_baselines</code> + <code>scan_critical_function_drift()</code>	sha256 per critical function
<code>npm run guard:all</code>	frontend ledger writes, deposit purpose, canonical tags, schema types, persona routes

8. Adding a new money flow

1. **Name the economic event.** What does the company own or owe afterwards that it did not before? If nothing, it is a reclassification and must be balance-sheet neutral.
2. **Pick accounts for both sides.** Reuse existing codes. A new code needs a `ledger_account_catalog` row with `section`, `nature`, `sort_order`.
3. **Add the mapping row(s)** — `ledger_scope`, `category`, `wallet_bucket` (null for bucket-agnostic), `account_code`, `debit_when`. Bucket-specific beats bucket-agnostic.
4. **Add the category to `ledger_category_allowlist()`**. Reproduce the existing entries verbatim — the function is replaced, not appended to.
5. **Check the resolver.** Will any override in §6.2 capture your group's shape? Shape rules do not look at your category.
6. **Check `balancedLedgerPost.ts`**. If your flow runs through one of the four functions in §7.3, the group must balance on **base mapping alone**, with no synthetics.
7. **Check the enforcement trigger.** Does your group carry a treasury category? If so it will **raise**, not log.
8. **Post through `create_ledger_transaction`** with an idempotency key.
9. **Verify:** group balances raw and resolved; `assert_money_path_intact()` still 17/17; no new rows in `ledger_mapped_balance_violations`.

9. Common mistakes

MISTAKE	CONSEQUENCE
Computing a resolver-affected account from <code>ledger_account_map</code>	Wrong balances, wrong conclusions. See §6.1
Changing <code>debit_when</code> to make a group balance	Re-states all history through that row; may invert the economics
Assuming a balanced ledger means correct accounting	A group can balance and still misclassify. See §10.3
Trusting E4 / the violations table as proof of integrity	Both have blind spots. §6.4, §7.1

MISTAKE	CONSEQUENCE
Impact-analysing a mapping change with SQL only	Misses <code>balancedLedgerPost.ts</code> . §7.3
Mapping a category without allowlisting it	Insert fails under <code>strict_mode</code>
Allowlisting without mapping	Leg posts and silently resolves to A9
Posting to E4 to force a balance	Forbidden. Fix the entry instead
Omitting <code>idempotency_key</code>	Retries double-post money

10. Open defects

All figures **2026-09-24**.

10.1 A2 conflates a company asset with customer money — *root defect*

ORIGIN	AMOUNT
Company-supplied (<code>merchant_float_deliveries</code> , <code>cfo_direct_credit</code> allocations)	2,141,755,733
User-supplied (agents' own <code>deposit_requests</code>)	1,715,709,823

A2 is an **asset**, L1 a **liability**, and a user's money can sit in either. Every Float ↔ Withdrawable move therefore crosses the balance sheet and mechanically creates a 2× artefact. The fix is to split A2 — company float stays, user-funded float moves to L1. §10.2 is downstream of this.

10.2 Balance sheet unbalanced

≈ **+1,012,040,798** (excess debits):

COMPONENT	GROUPS	RESIDUAL
Cross-bucket groups where the X4 synthetic fires	679 / 1,035	+809,735,372
Cross-bucket groups not gated	236 / 1,976	+215,505,426
<code>agent_advance_topups</code> (<code>rent_disbursement</code> →A1/CR + <code>agent_advance_credit</code> →L1/CR)	6	(13,000,000)

The first arose 2026-09-23 when the X4 counterpart shipped **without** its companion mapping change. Evidence gathered 2026-09-24 established Float ↔ Withdrawable is **economically neutral**: 595 of 1,036 groups are same-user bucket moves, float is not withdrawable, and total obligation is identical before and after. Correct treatment is **DR L1 / CR L1** — no X4, no A2.

10.3 Cash deposits — malformed but correctly reported

37 `personal_deposit` physical-cash groups (**333,255,000**) post four legs (DR A5 / CR A2 / CR L1 / CR L1) because the wallet leg became purpose-aware on 2026-09-07 and the platform legs did not. **The `agent_float_cash_offset` → L1 `override` rescues them** — they resolve to DR A5 / CR L1 and the published reports are correct. The 666,510,000 residual exists only in the trigger's view.

Forward fix deployed 2026-09-24, commit `f905c1182c` . **No historical correction is required or safe** — posting reversals would unbalance groups that currently balance.

10.4 A4 carries a credit balance

(1,268,864,881) — an asset with a credit balance, contaminated with corrections and test data. Excluded from receivables reporting.

10.5 L3 has never been used

Zero legs, ever. Partner Returns are expensed to X2 and credited straight to wallets (L1). The obligation is captured, but the Returns Payable line reads nil permanently.

10.6 L5 nets to zero against real obligation

34 legs, accrued and settled same-period. Operationally 16,157,500 accrued-unpaid commission plus 6,527,029 queued proxy commission sit with no liability account.

10.7 Other

- A5 = 754,737,090 of collected cash with no banking entry, against records stating "Deposited on bank"
- 933 self-approved deposits, 521,230,452 in 14 days (creator = approver)
- CFO debit obligations 1,034,280,305 across 1,077 rows — no identified account
- L2 is 997,215,780 above active portfolios
- merchandise_revenue unmapped → resolves to A9
- A15 redundant; A10/A4 overlap

11. Verification runbook

Cash position — get_treasury_cash_position() (A1 + A5). Authoritative. get_treasury_snapshot is superseded and can show false negatives.

Balance sheet — get_statement_of_financial_position(now()) . Requires cfo / ceo / coo / manager / financial_ops / super_admin / cto. Raises "Not authorised" when auth.uid() is NULL, which it is on a direct postgres connection.

Trial balance — a full sofp_ledger_legs(now()) pass exceeds the MCP gateway timeout. Hash-partition four ways:

```
HAVING abs(hashtext(COALESCE(gl.transaction_group_id, gl.id)::text)) % 4 = <0..3>
```

Subledger ties

ACCOUNT	SUBLEDGER
A7	promissory_notes where status='activated' — ties exactly; best canary
A10	agent_advances.outstanding_balance
A12 / A13	merchandise_recovery_plans.outstanding_balance
A14	credit_access_draws.outstanding_balance
L2	investor_portfolios.investment_amount
L1	wallets withdrawable + locked

Before changing any mapping row

1. Is the account resolver-affected? (§6.1)
2. grep -rn "<category>" supabase/functions/ (§7.3)
3. Will the group still balance on base mapping alone?
4. Does any affected group carry a treasury category? (§7.1)
5. Re-run the four-partition trial balance afterwards.

12. Reference: category → account

Complete ledger_account_map , 2026-09-24, 113 rows. * = bucket-agnostic. Bucket-specific rows win.

ACCT	SCOPE	BUCKET	CATEGORY	DEBIT_WHEN
A1	platform	*	agent_landlord_payout	cash_in

ACCT	SCOPE	BUCKET	CATEGORY	DEBIT_WHEN
A1	platform	*	cash_at_bank_reclass	cash_in
A1	platform	*	partner_capital_cash_received	cash_out
A1	platform	*	rent_disbursement	cash_in
A1	platform	*	treasury_bank_deposit	cash_in
A1	platform	*	verified_bank_cash_recognised	cash_in
A1	platform	*	wallet_deposit	cash_out
A1	platform	*	wallet_transfer	cash_out
A1	platform	*	wallet_withdrawal	cash_out
A2	platform	*	agent_facilitated_capital_receivable	cash_out
A2	platform	*	agent_float_cash_offset	cash_in
A2	wallet	*	agent_float_assignment	cash_in
A2	wallet	*	agent_float_deposit	cash_in
A2	wallet	*	agent_float_settlement	cash_in
A2	wallet	*	agent_float_topup	cash_in
A2	wallet	*	agent_float_used_for_rent	cash_in
A2	wallet	float	bucket_reclass_in	cash_in
A2	wallet	float	bucket_reclass_out	cash_in
A2	wallet	*	proxy_partner_withdrawal	cash_in
A2	wallet	*	rent_float_funding	cash_in
A2	wallet	*	rent_payment_for_tenant	cash_in
A3	bridge	*	fee_receivable_created	cash_in
A3	bridge	*	rent_plan_receivable_restatement	cash_in
A3	bridge	*	rent_receivable_created	cash_in
A3	platform	*	agent_float_used_for_rent	cash_in
A3	platform	*	pool_rent_deployment	cash_out
A3	platform	*	rent_repayment	cash_out
A3	platform	*	tenant_repayment	cash_out
A3	platform	*	tenant_repayment_collected	cash_in
A4	platform	*	agent_repayment	cash_out
A4	platform	*	wallet_deduction	cash_out
A4	platform	*	wallet_deduction_cash_payout_retraction	cash_out
A4	platform	*	wallet_deduction_general_adjustment	cash_out
A4	wallet	*	advance_repayment	cash_in
A5	platform	*	cash_in_transit_banked	cash_in
A5	platform	*	cash_receipt_in_transit	cash_in
A6	bridge	*	landlord_receivable_created	cash_in
A6	platform	*	landlord_receivable_collected	cash_out

ACCT	SCOPE	BUCKET	CATEGORY	DEBIT_WHEN
A7	bridge	*	partner_receivable_created	cash_in
A7	platform	*	partner_receivable_collected	cash_out
A8	platform	*	agent_float_assignment	cash_in
A8	platform	*	agent_float_cycle_settled_to_bank	cash_in
A8	platform	*	agent_float_deposit	cash_in
A8	platform	*	agent_float_funding	cash_in
A8	platform	*	agent_float_settlement	cash_in
A8	platform	*	agent_float_topup	cash_in
A9	bridge	*	orphan_reassignment	cash_in
A9	bridge	*	orphan_reversal	cash_in
A10	bridge	*	agent_advance_receivable_opening	cash_in
A10	platform	*	agent_advance_repayment	cash_out
A11	bridge	*	agent_access_fee_receivable_opening	cash_in
A12	bridge	*	merchandise_recovery_receivable_opening	cash_in
A12	platform	*	merchandise_recovery_repayment	cash_out
A12	platform	*	smartphone_advance_receivable	cash_out
A13	bridge	*	bike_recovery_receivable_opening	cash_in
A13	platform	*	bike_recovery_repayment	cash_out
A14	bridge	*	credit_draw_receivable_opening	cash_in
A14	platform	*	credit_draw_repayment	cash_out
A15	bridge	*	merchandise_credit_sale_receivable_opening	cash_in
A16	bridge	*	service_centre_advance_receivable_opening	cash_in
A17	bridge	*	service_centre_receivable_opening	cash_in
A18	bridge	*	tenant_service_charge_receivable_opening	cash_in
A19	bridge	*	business_advance_receivable_opening	cash_in
E1	platform	*	pool_capital_received	cash_out
E1	platform	*	share_capital	cash_out
E3	platform	*	balance_correction	cash_out
E3	platform	*	historical_balance_reseed	cash_out
E3	platform	*	receivable_restatement_equity	cash_in
L1	platform	*	cash_custody_payable	cash_in
L1	platform	*	roi_wallet_credit	cash_out
L1	wallet	withdrawable	bucket_reclass_in	cash_in
L1	wallet	withdrawable	bucket_reclass_out	cash_in
L1	wallet	float	partner_funding	cash_out
L1	wallet	*	partner_funding	cash_out
L1	wallet	*	supporter_rent_fund	cash_out

ACCT	SCOPE	BUCKET	CATEGORY	DEBIT_WHEN
L1	wallet	withdrawable	tenant_rent_settlement	cash_out
L2	bridge	*	partner_funding	cash_out
L2	bridge	*	supporter_facilitation_capital	cash_out
L2	platform	*	partner_funding	cash_out
L2	platform	*	partner_receivable_capital	cash_in
L2	platform	*	roi_reinvestment	cash_out
L3	platform	*	partner_reward_accrued	cash_in
L4	platform	*	landlord_receivable_obligation	cash_in
L5	platform	*	agent_commission_accrued	cash_in
L5	platform	*	agent_commission_settled	cash_out
L6	platform	*	pending_portfolio_topup	cash_out
L7	platform	*	treasury_allocated	cash_in
L7	platform	*	treasury_fee_drawdown	cash_out
L7	platform	*	treasury_fee_recognised	cash_in
R1	platform	*	access_fee_collected	cash_out
R1	platform	*	listing_rejection_recovery	cash_out
R1	platform	*	registration_fee_collected	cash_out
R1	platform	*	tenant_access_fee	cash_out
R1	platform	*	treasury_net_revenue	cash_in
X1	platform	*	agent_bonus · equipment_expense · general_admin_expense · interest_expense · manager_credit · marketing_expense · payroll_expense · platform_expense · research_development_expense · salary_payout · supporter_platform_rewards · tax_expense	cash_out
X2	platform	*	roi_expense	cash_out
X3	bridge	*	agent_commission	cash_out
X3	platform	*	agent_commission_earned	cash_out
X3	platform	*	agent_commission_payable	cash_out
X3	platform	*	agent_commission_payout	cash_out
X4	platform	*	platform_loss_writeoff	cash_out
X5	platform	*	merchant_oop_reimbursement	cash_out

13. Glossary

TERM	MEANING
Leg	One row in <code>general_ledger</code>
Group	All legs sharing a <code>transaction_group_id</code> ; the unit that must balance
Scope	<code>wallet</code> · <code>platform</code> · <code>bridge</code> — see §2.1

TERM	MEANING
Bucket	<code>withdrawable</code> · <code>float</code> · <code>advance</code> · <code>locked</code>
<code>debit_when</code>	The <code>direction</code> value that makes a leg a debit, per mapping row
Base mapping	Account/side from <code>ledger_account_map</code> before overrides
Override	Shape-keyed rewrite of account or direction inside the resolver
Synthetic leg	A leg the resolver invents; exists in no table
Gated group	A group matching a synthetic's shape condition
<code>raw_net</code>	$\sum \text{cash_in} - \sum \text{cash_out}$ on raw directions, ignoring mapping
Resolver	<code>sofp_ledger_legs(as_at)</code>
Subledger	The operational table behind an account (e.g. <code>promissory_notes</code> → A7)